IOT SMART IRRIGATION SYSTEM

Mini Project Report

Submitted in partial fulfillment of the
Requirements for the award of the Degree of

BACHELOR OF SCIENCE (COMPUTER SCIENCE)

By

Mr. Ram Niraj

Seat Number : 510513059

Under the esteemed guidance of

Prof. Mrs. Sahaya Jeicy

Department of Computer Science

R.P. Institute of Hospitality & Management

DEPARTMENT OF COMPUTER SCIENCE

R.P. INSTITUTE OF HOSPITALITY & MANAGEMENT

(Affiliated to University of Mumbai)

MUMBAI - 401105

MAHARASHTRA

(2026-2027)

PROFORMA FOR THE APPROVAL OF PROJECT PROPOSAL

ERN NO.: _____

SEAT NUMBER: 510513059

1. **Name of the student**
   Ram Niraj

2. **Title of the Project**
   IoT Smart Irrigation System

3. **Name of the Guide**
   Prof. Mrs. Sahaya Jeicy

4. **Teaching experience of the Guide**
   _____

5. Is this your first submission? Yes [ ] No [ ]

Signature of the student

Signature of the Guide

Date: _____

Date: _____

Signature of the Coordinator: _____

Date: _____

R.P. INSTITUTE OF HOSPITALITY & MANAGEMENT

(Affiliated to University of Mumbai)

MUMBAI - MAHARASHTRA

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that the project entitled “IoT Smart Irrigation System” is a bonafide work of Mr. Ram Niraj, Seat Number 510513059, submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Science in Computer Science from the University of Mumbai. The project report describes a web-based educational demonstration of a smart irrigation concept using HTML, CSS and JavaScript, together with a proposed IoT hardware architecture.

Internal Guide	Coordinator / HOD	External Examiner
Date: _____	Date: _____	Date: _____

College Seal

ABSTRACT

The IoT Smart Irrigation System project presents a simple web-based educational interface to explain how connected sensing can support efficient watering. In a smart irrigation arrangement, a soil-moisture sensor measures the condition of the soil and sends a reading to a controller such as an ESP32 or Arduino. The controller compares the reading with a selected threshold and can activate a relay and water pump when irrigation is required. When the soil reaches an adequate moisture level, the pump can be switched off. Optional temperature and humidity readings and Wi-Fi connectivity can support remote monitoring.

The supplied implementation is a single-page website developed with HTML, CSS and JavaScript. It includes an overview, problem statement, feature cards, system workflow, hardware and software requirements, project objectives, references, and an interactive moisture demo. In the demo, the illustrative threshold is 40 percent: readings below 40 percent show the pump as ON, while readings at or above 40 percent show the pump as OFF. The website uses responsive styling and navigation links to make the material easier to explore on a mobile device.

The current website is a visual prototype only. Its interactive demo changes displayed values in the browser and is not connected to physical sensors, a relay, a pump, a cloud database or a live IoT platform. The report therefore documents both the implemented website and the proposed physical system, clearly separating demonstrated software behavior from hardware that could be developed in a future phase. The project demonstrates basic web development, interface design, decision logic, requirements analysis and technical documentation while introducing the practical role of IoT in water conservation.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Prof. Mrs. Sahaya Jeicy, and the faculty members of the Department of Computer Science for their guidance, encouragement and support during the preparation of this project report.

I am thankful to R.P. Institute of Hospitality & Management for providing the academic environment in which I could study the concepts of computer science, web development and the Internet of Things. The project helped me connect classroom learning with a practical educational example of water-aware irrigation.

I also thank my friends and classmates for their suggestions and encouragement. Finally, I am grateful to my family for their patience and support throughout the work.

This report records the supplied website implementation and its conceptual hardware design. Any future physical prototype will require additional development, component testing and appropriate electrical safety measures.

Ram Niraj
Seat Number 510513059

DECLARATION

I hereby declare that the project entitled “IoT Smart Irrigation System” has been prepared by me as part of the academic requirements for the Bachelor of Science (Computer Science) programme. The report presents the educational website supplied with the project, its user interface, its client-side moisture demonstration, its conceptual system design, and its proposed areas for future development.

I further declare that the documentation has been prepared for academic submission and that the sources consulted for technical background are listed in the References section. The interactive moisture demo is a browser-based simulation and should not be interpreted as evidence that physical irrigation hardware has been built or connected.

Name: Ram Niraj

Seat Number: 510513059

Signature: _____

Date: _____

TABLE OF CONTENTS

Chapter 1 - Introduction **10**
1.1 Introduction 10
1.2 Background of the Project 10
1.3 Problem Statement 10
1.4 Aim of the Project 10
1.5 Objectives 10
1.6 Scope of the Project 11
1.7 Importance of the Project 11
Chapter 2 - Literature Survey **12**
2.1 Introduction 12
2.2 Traditional Irrigation Practices 12
2.3 Existing Smart Irrigation Concepts 12
2.4 Limitations of Existing Practices 12
2.5 Proposed System 12
2.6 Feasibility Study 13
Chapter 3 - Feature Overview **14**
3.1 Home Page and Navigation 14
3.2 Project Overview 14
3.3 Key Features 14
3.4 Working Process 14
3.5 Interactive Moisture Demo 14
3.6 Hardware and Software Sections 15
3.7 References and Footer 15
3.8 Navigation Summary 15

TABLE OF CONTENTS (CONTINUED)

Chapter 4 - System Requirements and Specifications **16**
4.1 Functional Requirements 16
4.2 Non-Functional Requirements 16
4.3 Hardware Requirements 17
4.4 Software Requirements 17
4.5 Constraints and Assumptions 17
Chapter 5 - System Design **18**
5.1 Design Approach 18
5.2 System Architecture 18
5.3 Data Flow Diagram 18
5.4 Decision Flowchart 20
5.5 Use-Case View 20
5.6 Interface Design 21
Chapter 6 - Implementation and Testing **22**
6.1 Development Environment 22
6.2 HTML, CSS and JavaScript 22
6.3 Responsive Design 22
6.4 Demo Logic 22
6.7 Test Cases and Results 29
6.8 Development Status Summary 29
Chapter 7 - Challenges and Future Scope **30**
7.1 Development Challenges 30
7.2 Current Limitations 30
7.3 Future Scope 30
8.1 Conclusion 31
8.2 Learning Outcomes 31
8.3 Final Remarks 31
References **32**
Appendix A - Source Code Listing **33**
6.6 Output Screenshots 22
6.5 Hardware Integration Plan 22
Chapter 8 - Conclusion **31**
7.4 Safety and Reliability Considerations 30

LIST OF FIGURES, TABLES AND ABBREVIATIONS

List of Figures

- Figure 5.1 - System architecture of the implemented website and proposed IoT hardware
- Figure 5.2 - Level-0 data flow of the website demonstration
- Figure 5.3 - Moisture threshold decision flowchart
- Figure 5.4 - Use-case view of the website
- Figure 6.1 - Project home page and hero section
- Figure 6.2 - Intelligent watering highlight card
- Figure 6.3 - Overview, introduction and problem statement
- Figure 6.4 - Key feature cards
- Figure 6.5 - Environment sensors, remote access and use case
- Figure 6.6 - How it works section and process flow
- Figure 6.7 - Moisture operation explanation and demo
- Figure 6.8 - Hardware and software requirements
- Figure 6.9 - Hardware and software requirements
- Figure 6.10 - Software requirements and objectives
- Figure 6.11 - Advantages and conclusion
- Figure 6.12 - References and project footer

List of Tables

- Table 4.1 - Functional requirements
- Table 4.2 - Non-functional requirements
- Table 4.3 - Hardware component requirements
- Table 4.4 - Software requirements
- Table 6.1 - Test cases and expected results
- Table 6.2 - Development status summary

Abbreviations

Abbreviation	Meaning
IoT	Internet of Things
UI	User Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
MCU	Microcontroller Unit
ESP32	Espressif ESP32 microcontroller family
DFD	Data Flow Diagram
Wi-Fi	Wireless Local Area Network connectivity
DHT11 / DHT22	Temperature and humidity sensor modules
IDE	Integrated Development Environment

CHAPTER 1 - INTRODUCTION

1.1 Introduction

Water is essential for agriculture, and irrigation needs to be applied at the right time and in a suitable quantity. Traditional watering often depends on fixed schedules or manual observation. If the soil is already moist, watering may be unnecessary; if the soil dries out between checks, a crop may not receive water when it needs it. These challenges make sensor-based irrigation a useful example of automation in agriculture.

The Internet of Things (IoT) allows physical devices such as sensors, controllers and actuators to exchange measurements or commands. In a smart irrigation arrangement, a soil-moisture sensor estimates the moisture condition of the soil. A controller interprets the reading and can switch a pump through a relay. Connectivity can be added to report readings to a phone or web dashboard.

1.2 Background of the Project

Smart irrigation combines measurement and control so that watering decisions are based on the condition of the soil rather than time alone. A small controller such as an ESP32 or Arduino can read a sensor and run a threshold rule. A relay module can control a suitable pump circuit, while optional temperature and humidity sensing may provide additional context. Wi-Fi-enabled monitoring can let a user review conditions without being continuously near the field.

The supplied project material contains a single-page website that explains this concept and provides an interactive visual demonstration. The report therefore covers two connected layers: the software that is actually present in the HTML file and the physical IoT architecture described as a proposed implementation. The two are not treated as identical.

1.3 Problem Statement

Manual irrigation can use more water than needed, miss periods when plants need water, and require repeated human observation. A student or new user may also find it difficult to understand how sensor readings are translated into an irrigation decision. The project addresses this educational problem by presenting the main components, workflow and decision rule in a clear browser-based interface.

1.4 Aim of the Project

The aim is to develop an accessible, responsive website that explains the working principle of an IoT-based smart irrigation system and demonstrates a sample moisture-threshold decision through a browser-based simulation.

1.5 Objectives

- To explain the role of soil-moisture sensing in irrigation decisions.
- To describe how a microcontroller, relay module and water pump can work together.
- To illustrate automatic pump status using a simple moisture threshold rule.
- To present hardware and software requirements in an organized web interface.
- To introduce IoT monitoring and the potential to reduce unnecessary watering and manual effort.

1.6 Scope of the Project

The current scope includes a single-page educational website, navigation between sections, project and component descriptions, a conceptual system workflow, a moisture demo controlled by on-page buttons, and links to selected technical documentation. It is suitable for demonstration of web development and the basic logic behind an irrigation controller.

Physical wiring, sensor calibration, live device communications, user authentication, a cloud database, mobile application deployment and farm-scale performance measurement are outside the supplied implementation. These can be considered future work after appropriate hardware and testing are available.

1.7 Importance of the Project

The project links computer science with a practical environmental application. It introduces interface design, responsive layout, client-side programming, simple decision logic and technical documentation, while highlighting water conservation as an intended benefit of sensor-based control. Its educational prototype status makes the concept easier to demonstrate without claiming that hardware has already been deployed.

Project at a Glance

Item	Summary
Implemented deliverable	Single-page website with interactive browser demo
Core technologies	HTML, CSS and JavaScript
Illustrative threshold	Below 40%: pump ON; at or above 40%: pump OFF
Physical hardware	Proposed only; not connected to the website
Main intended benefit	Explain sensor-based irrigation and water conservation

CHAPTER 2 - LITERATURE SURVEY

2.1 Introduction

A literature survey reviews the general approaches related to the project and identifies how the proposed work fits within them. This report uses broad technical concepts and official documentation for microcontrollers, Arduino development and IoT analytics as background. It does not claim to be a controlled agricultural field study.

2.2 Traditional Irrigation Practices

Traditional irrigation may be operated manually or according to a fixed schedule. Manual control gives the operator direct judgment but requires regular attention and depends on observation. Schedule-based watering is simple to operate, yet a fixed schedule does not always reflect changes in soil moisture, weather, crop type or growing stage. Both approaches can result in excess or insufficient watering when conditions change.

2.3 Existing Smart Irrigation Concepts

Many smart-irrigation concepts use sensors, a microcontroller and an actuator. Soil-moisture readings can be compared with a selected threshold; when the soil is dry, the controller requests irrigation, and when moisture is adequate, irrigation is stopped. An IoT-enabled controller may additionally publish values over a network so that the user can review readings or receive status information remotely.

Common implementation choices include Arduino-compatible boards or ESP32-class controllers, capacitive or resistive soil-moisture sensors, a relay or motor driver appropriate to the load, a pump and tubing. Environmental sensors such as the DHT11 or DHT22 can measure temperature and humidity. Actual component selection must consider operating voltage, current draw, sensor range, electrical isolation and the environment in which the system will operate.

2.4 Limitations of Existing Practices

- Manual operation depends on the availability and consistency of the person managing irrigation.
- Fixed-time schedules may not adapt to changing soil or weather conditions.
- Poor sensor calibration may cause early or delayed pump activation.
- Wireless connectivity may be unavailable or unreliable in some agricultural locations.
- Unsafe wiring or an incorrectly rated relay can damage equipment or create electrical hazards.

2.5 Proposed System

The project website explains a proposed sensor-based architecture and shows its logic using a small browser demo. The educational rule uses an example threshold of 40 percent. Values below the threshold are displayed as dry soil with the pump status ON; values at or above the threshold are displayed as sufficient moisture with pump status OFF. The threshold is illustrative rather than a calibrated agricultural value and would need adjustment for a real sensor and crop.

The current website includes no connection from JavaScript to a physical controller. In a future implementation, a controller could read the sensor, perform the threshold test, activate a correctly rated relay, and optionally send telemetry to an IoT service such as ThingSpeak or Blynk. That phase would require firmware, wiring, calibration and end-to-end testing.

2.6 Feasibility Study

Feasibility area	Assessment
Technical	The current website is technically simple and runs in a modern browser. Physical automation is feasible in principle, but is not implemented in the supplied HTML.
Economic	The educational website can be demonstrated at low cost. Hardware cost would depend on the controller, sensor, pump, power supply, enclosure and safe switching components.
Operational	Section links and buttons are straightforward to use. A real installation would require maintenance, calibration and a safe manual override.
Schedule	A small front-end demonstration is feasible as a student project. A tested physical prototype requires additional time for assembly and measurements.
Environmental	The concept aims to reduce unnecessary irrigation; savings have not been measured by this simulation.

CHAPTER 3 - FEATURE OVERVIEW

3.1 Home Page and Navigation

The landing section introduces the project with the message “Grow smarter. Save every drop.” It identifies the topic as IoT, smart agriculture and water conservation and provides two links for moving to the working explanation or project overview. The name and seat number appear in the hero section so that the project owner is visible when the site is presented.

The navigation bar contains links to About, Features, Working, Hardware and References. These links use in-page anchor targets rather than separate server pages. The sticky header remains visible while the visitor scrolls through the long page, making navigation easier on a small screen.

3.2 Project Overview

The overview section contains an introduction and a problem statement. The introduction explains how soil and environmental conditions can be monitored and how the controller can turn a pump ON or OFF based on moisture. The problem statement describes the risks of over-watering, under-watering, wasted water and unnecessary pump operation.

3.3 Key Features

Feature	Description in the website
Automatic irrigation	Explains that a pump can be switched ON or OFF based on a soil-moisture reading.
Live monitoring	Describes showing sensor conditions in a web or mobile application.
Environment sensors	Lists DHT11 or DHT22 as optional temperature and humidity sensors.
Remote access	Introduces Wi-Fi and the possibility of remote monitoring via an IoT platform.
Less waste	Explains the intended reduction of unnecessary watering and pump operation.
Flexible use	Identifies gardens, farms, greenhouses and small fields as conceptual use cases.

3.4 Working Process

The illustrated hardware workflow is: soil-moisture sensor -> ESP32 or Arduino controller -> relay module -> water pump -> irrigation area. The sensor supplies a reading; the controller interprets that reading; the relay acts as an electrically controlled switch; and the pump moves water through the irrigation pipe. In a complete IoT system, the controller may also send sensor values through Wi-Fi for remote monitoring.

3.5 Interactive Moisture Demo

The demo presents a percentage value, a progress bar, a status message and the words PUMP ON or PUMP OFF. Three buttons are provided: “Make soil drier”, “Add moisture” and “Reset demo”. In the supplied JavaScript, the first button sets the illustrative value to 25 percent, the second sets it to

75 percent and the reset button restores 42 percent. The displayed pump state is computed from the 40 percent threshold.

3.6 Hardware and Software Sections

The hardware table lists an ESP32 or Arduino Uno, soil-moisture sensor, optional DHT11/DHT22, relay module, water pump and pipe, power supply, jumper wires, breadboard and Wi-Fi. The software list names Arduino IDE, C/C++ for Arduino programming, Blynk or ThingSpeak, optional Firebase, and a web browser or mobile app. In the current website only the front-end HTML, CSS and JavaScript are implemented; the other components are listed as requirements for the proposed physical version.

3.7 References and Footer

The references section links to official documentation from Espressif Systems, Arduino, ThingSpeak and Blynk. The footer repeats the project title, name, seat number and semester, and explicitly notes that the demo controls do not operate physical hardware. This disclaimer is an important distinction for accurate project presentation.

3.8 Navigation Summary

Navigation link	Target content
About	Introduction and problem statement
Features	Automatic irrigation, monitoring, optional sensors and remote access
Working	Proposed sensor-to-pump workflow and operating cases
Hardware	Hardware/software lists and project objectives
References	Official documentation links

CHAPTER 4 - SYSTEM REQUIREMENTS AND SPECIFICATIONS

4.1 Functional Requirements

Functional requirements describe what a system should do from the user's point of view. The current website has the following requirements and implemented behaviors:

ID	Requirement	Status in supplied website
FR-01	Display project identity and introduction	Implemented
FR-02	Navigate to About, Features, Working, Hardware and References	Implemented with anchor links
FR-03	Show overview and problem statement	Implemented
FR-04	Describe features and hardware components	Implemented as static page content
FR-05	Change moisture demo value by button action	Implemented in JavaScript
FR-06	Calculate displayed demo status from threshold	Implemented in JavaScript
FR-07	Connect to physical sensor / relay / pump	Not implemented
FR-08	Transmit live values to an IoT cloud dashboard	Not implemented
FR-09	Provide links to selected technical references	Implemented

4.2 Non-Functional Requirements

Requirement	Description
Usability	Clear labels, recognizable sections and simple controls.
Responsiveness	CSS media queries adapt the layout for smaller mobile screens.
Compatibility	Uses standard HTML, CSS and JavaScript features expected in modern browsers.
Performance	Single-page content with no server request required for the demo.
Accessibility	Headings and descriptive labels improve scanning; further keyboard and screen-reader testing is recommended.
Reliability	The local demo logic is deterministic; it does not provide live hardware reliability guarantees.
Maintainability	HTML, CSS and JavaScript are contained in one source file, which is easy to distribute but can become harder to manage as the project grows.

4.3 Hardware Requirements

Component	Purpose	Implementation status
ESP32 / Arduino Uno	Read sensor input and run controller logic	Listed for a future prototype
Soil-moisture sensor	Measure moisture in soil	Not connected to the webpage
DHT11 / DHT22	Measure temperature and humidity	Optional; not connected
Relay module	Switch a pump circuit	Conceptual only
Water pump and pipe	Deliver water to plants	Conceptual only
Power supply	Provide suitable power	Must be selected for actual components
Breadboard and jumper wires	Prototype connections	Not demonstrated in screenshots
Wi-Fi / IoT service	Publish readings for remote monitoring	Described only

4.4 Software Requirements

Software / platform	Purpose and status
Web browser	Runs the website and its client-side demo; implemented.
HTML5	Structures the page sections, cards, tables and links; implemented.
CSS3	Defines layout, colors, typography, responsive rules and visual styling; implemented.
JavaScript	Updates moisture value, progress bar, message and pump status; implemented.
Arduino IDE and C/C++	Would be used for embedded firmware; not part of the current website.
Blynk / ThingSpeak / Firebase	Possible future options for remote telemetry; no live integration is present.

4.5 Constraints and Assumptions

- The current deliverable is a single HTML file that can be opened in a browser; it does not require a backend to show the demo.
- The moisture percentage is an illustrative browser value, not a direct reading from a calibrated sensor.
- The 40 percent threshold is an example for demonstrating logic and is not universally appropriate for every crop or soil type.
- The described hardware architecture assumes components are correctly rated and wired by a competent person.
- The report does not claim measured water savings, crop-yield improvement or field-test results.

CHAPTER 5 - SYSTEM DESIGN

5.1 Design Approach

The project follows a simple presentation-first design. A visitor opens a single page, reads the overview, reviews the feature and hardware sections, and can try the moisture demo. The site separates static educational content from the small interactive component: HTML provides document structure, CSS controls presentation, and JavaScript updates the demo state in response to button clicks.

5.2 System Architecture

Figure 5.1 distinguishes the implemented browser layer from a proposed physical IoT layer. The current implementation ends at the rendered website and its simulated status. A physical design would begin with a soil-moisture sensor, pass readings to a controller, and use a suitable relay and pump to irrigate plants. The dashed conceptual separation is intentional because the supplied HTML has no hardware interface.

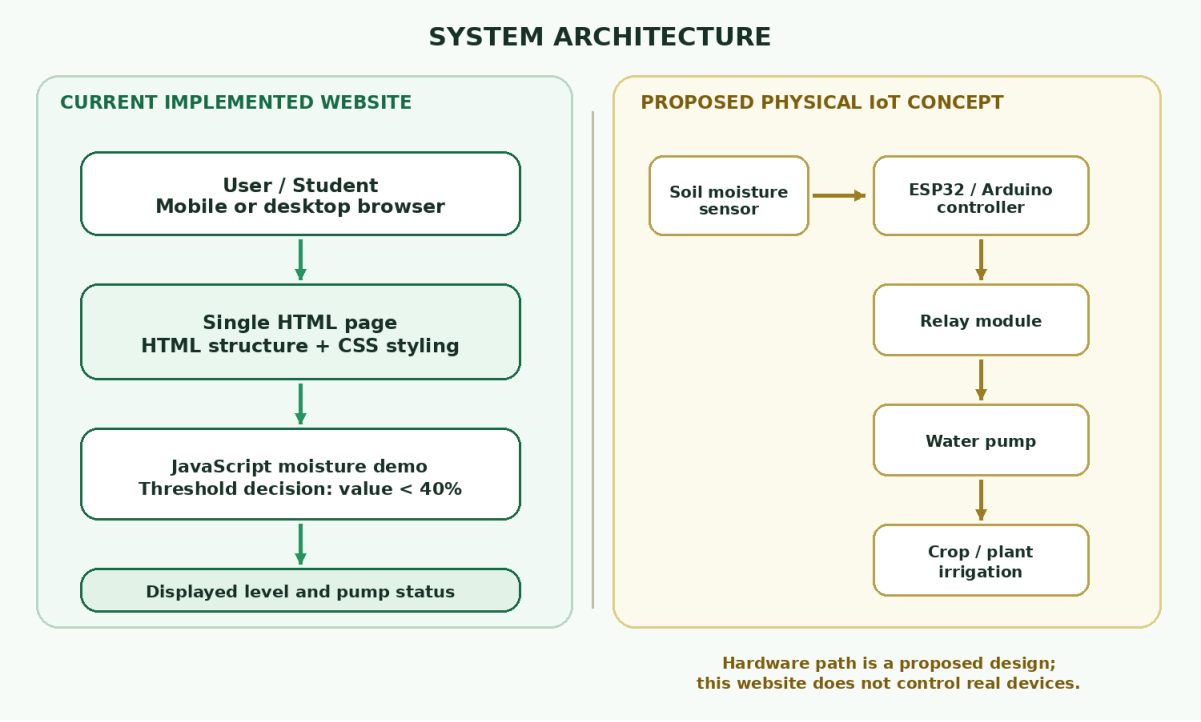


Figure 5.1: Implemented website and proposed physical IoT architecture

5.3 Data Flow Diagram

At level 0, the visitor supplies navigation actions or demo button clicks. The website interface displays static project content and forwards a demo action to JavaScript. The demo logic updates the moisture state and then returns a presentation state to the page: percentage, progress-bar width, condition message and pump-status text. The static sections do not depend on a server or database.

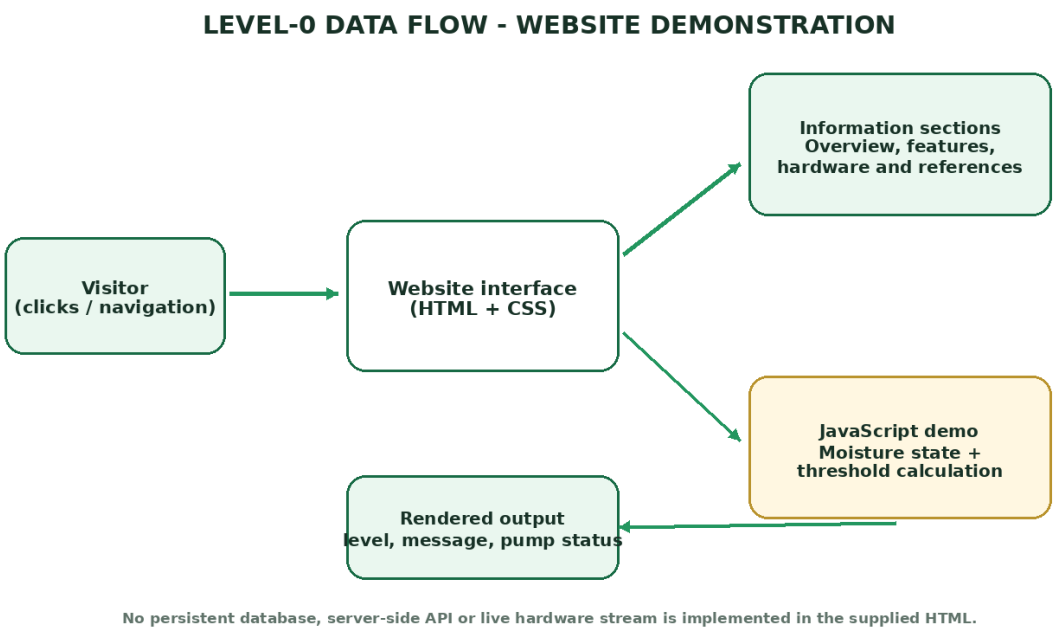


Figure 5.2: Level-0 data flow of the website demonstration

5.4 Decision Flowchart

The demo initializes the moisture value to 42 percent. When a button is clicked, JavaScript assigns a new illustrative value. The code computes a boolean condition equivalent to “value is less than 40”. If the condition is true, it displays that the soil is dry and the pump is ON. Otherwise, it displays that moisture is sufficient and the pump is OFF. The initial 42 percent value therefore displays the OFF state in the current source code.

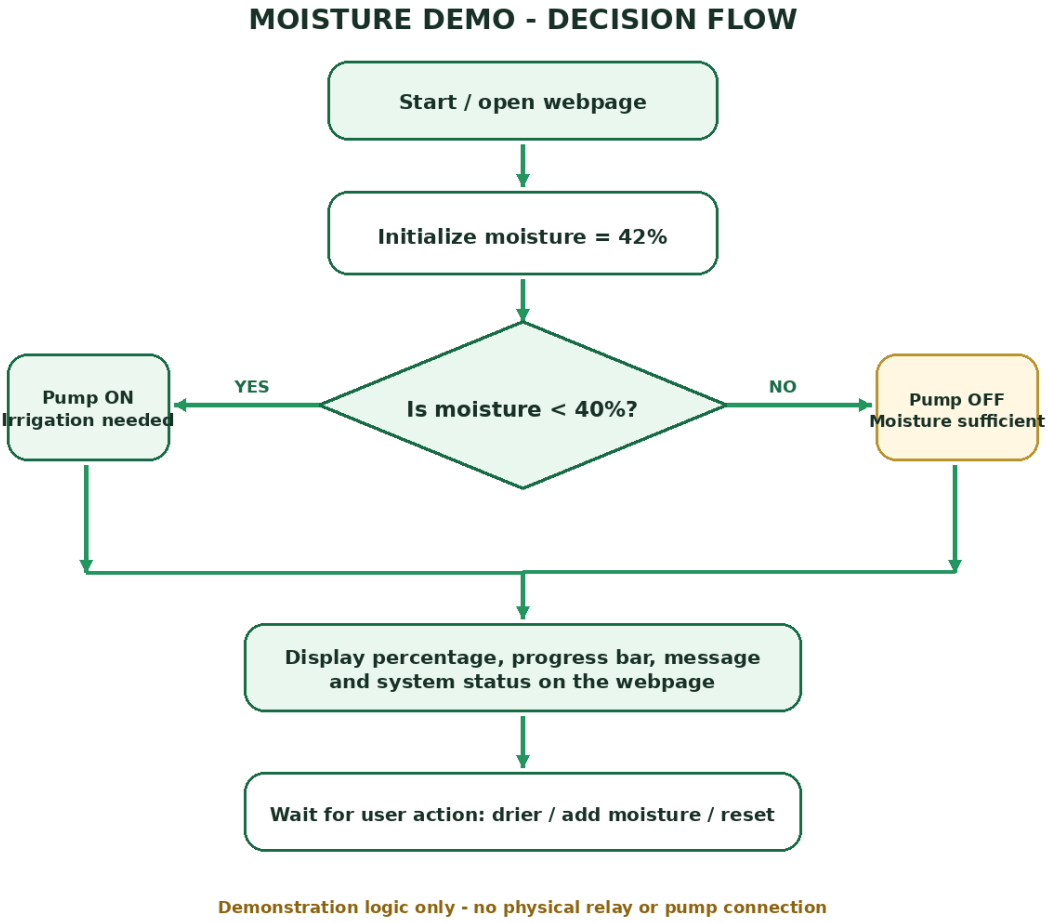


Figure 5.3: Moisture-threshold decision flow for the browser demo

5.5 Use-Case View

The website has one primary actor: a visitor or student. The actor can view the project overview, read the features and workflow, review the proposed hardware and software, interact with the moisture demo, and open external references. The current design has no registration, administration or authenticated roles.

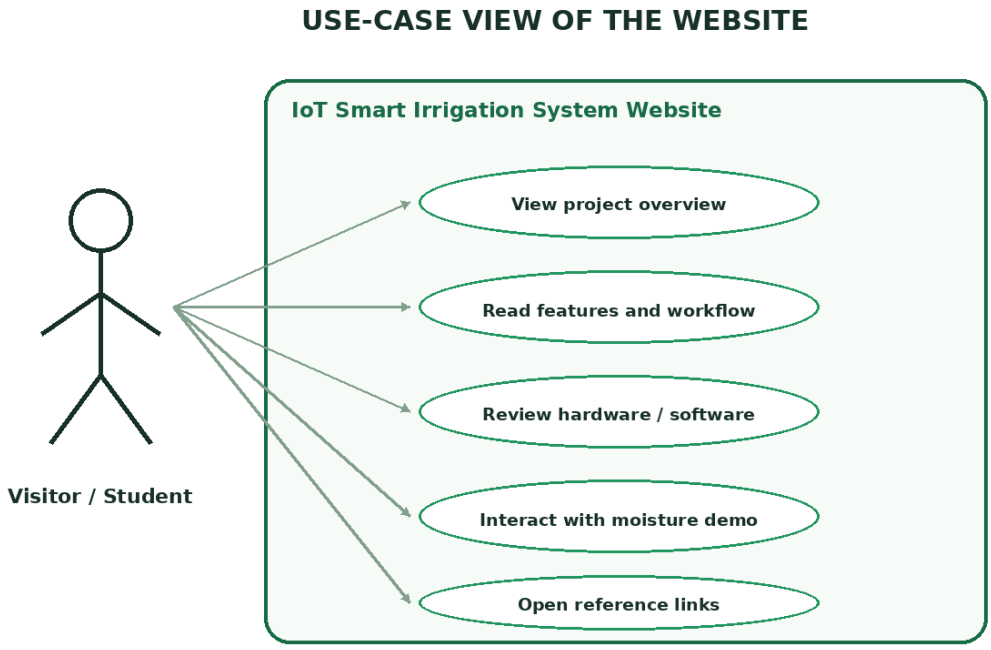


Figure 5.4: Use-case view of the educational website

5.6 Interface Design

The website uses a green and mint palette to reinforce the plant and water theme. A sticky gradient header contains the brand and in-page links. Cards separate explanatory topics, while a dark-green demo panel emphasizes the numerical reading and pump status. Rounded buttons make demo actions easy to identify. CSS grid and flex layouts organize content on larger screens, with media queries adapting the hero and card layouts for mobile widths.

CHAPTER 6 - IMPLEMENTATION AND TESTING

6.1 Development Environment

The supplied website is a standalone HTML file. It can be opened in a current web browser without installing a development server. The screenshots supplied for documentation show the page in a mobile browser. Editing can be performed in a text editor or code editor, while browser refresh is sufficient to review changes. A hardware prototype would require a separate embedded development environment such as Arduino IDE.

6.2 HTML, CSS and JavaScript

HTML defines the header, navigation, hero section, content sections, tables and footer. CSS provides the page colors, spacing, cards, button styles, responsive grid/flex behavior and mobile media queries. JavaScript defines the setMoisture(value) function, which updates the displayed percentage, progress-bar width, condition message, pump label and demo status message. The function is called on initial page load and from each button.

6.3 Responsive Design

The CSS includes a breakpoint at widths below 800 pixels to stack the hero content, reduce the number of card columns, and allow the navigation links to wrap. At widths below 540 pixels, the card grid becomes a single column and section padding is reduced. These rules are visible in the submitted mobile screenshots and help avoid a desktop-only layout.

6.4 Demo Logic

The demo starts at 42 percent. “Make soil drier” changes the value to 25 percent; “Add moisture” changes it to 75 percent; “Reset demo” restores 42 percent. Each action runs the same threshold check and updates all visual indicators. The demo illustrates conditional logic and state changes in a webpage, but its numeric values are manually set examples rather than sensor measurements.

6.5 Hardware Integration Plan

To convert the concept into a working physical prototype, a future version would need to read a moisture sensor from the selected controller, calibrate the sensor readings using the intended soil, control the pump through an appropriately rated and isolated switching circuit, and verify the full electrical arrangement before operation. Optional telemetry could then be sent to an IoT platform. The browser demo should remain clearly labeled as simulation until this integration is actually completed.

6.6 Output Screenshots

The following screen captures document the user interface delivered with the project. They show the home page, overview, feature cards, working process, demo, requirements, conclusion and references. They are screenshots of the website content and are not test evidence of a physical sensor or pump.

PROJECT OUTPUT - HOME PAGE AND OVERVIEW

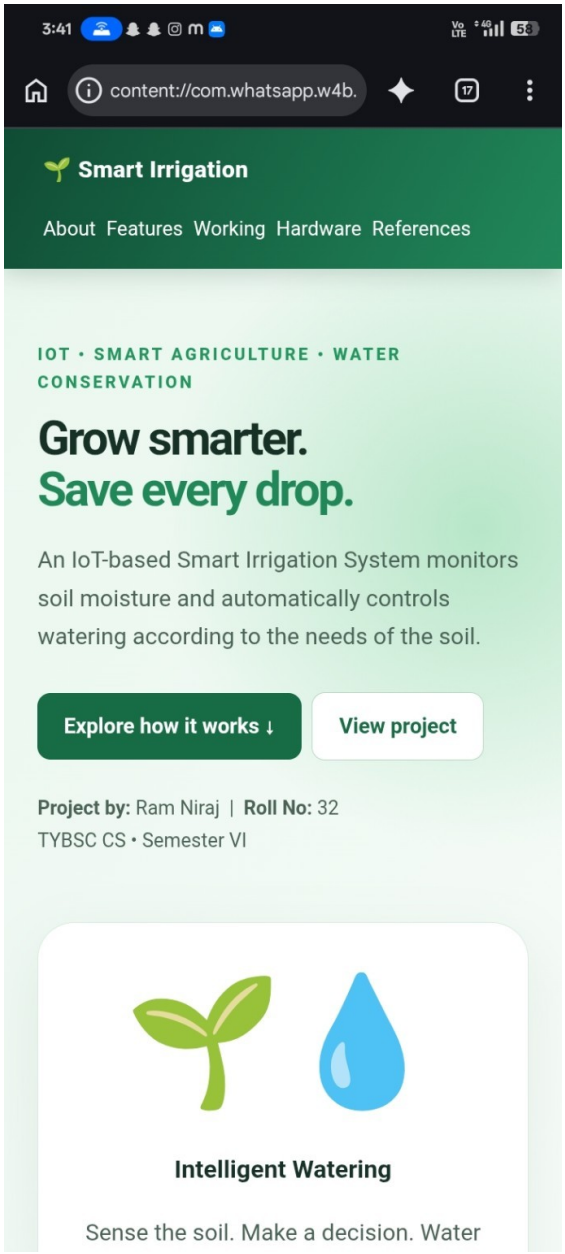


Figure 6.1: Home page and hero section

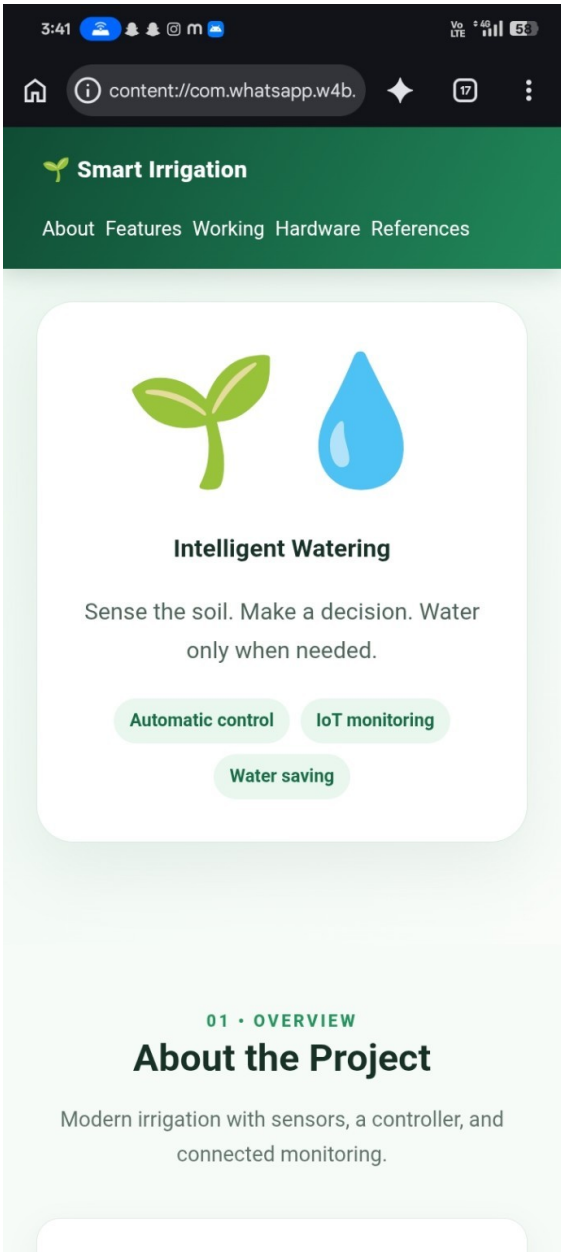


Figure 6.2: Intelligent watering card

PROJECT OUTPUT - OVERVIEW AND FEATURES

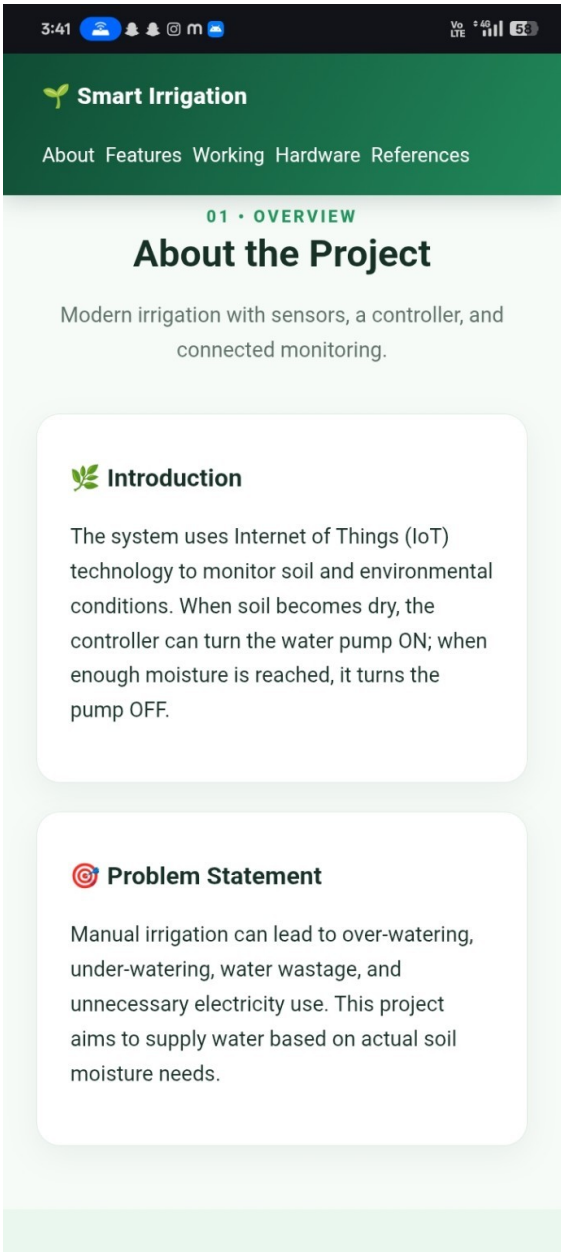


Figure 6.3: Introduction and problem statement

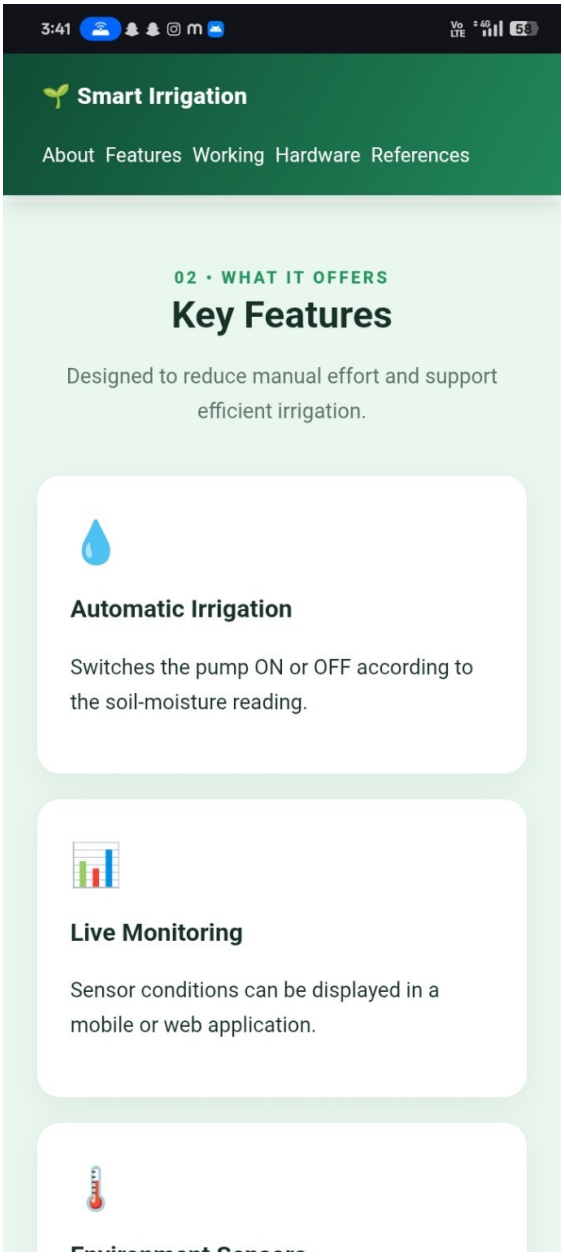


Figure 6.4: Key feature cards

PROJECT OUTPUT - SENSORS AND WORKFLOW

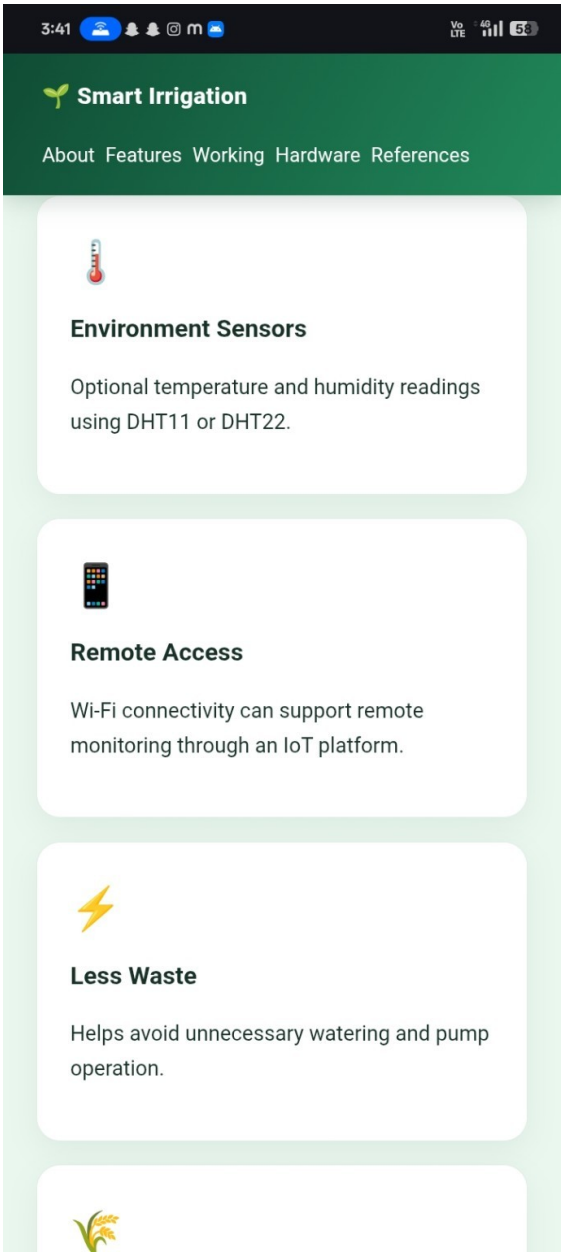


Figure 6.5: Environment sensors and remote access

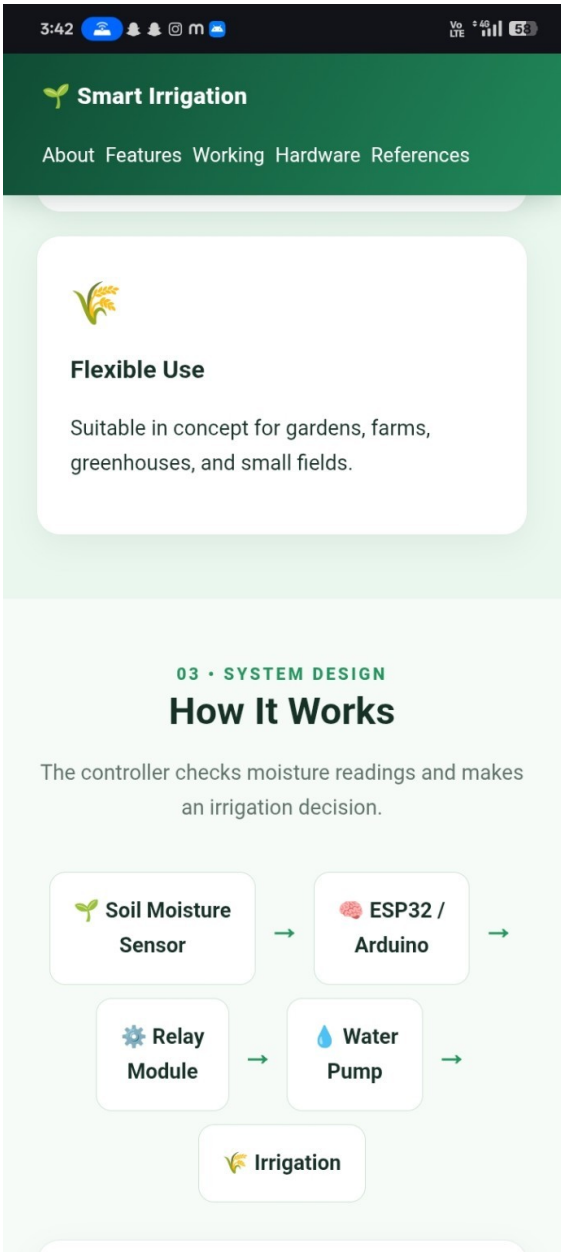


Figure 6.6: Hardware workflow diagram

PROJECT OUTPUT - WORKING CONCEPT AND DEMO

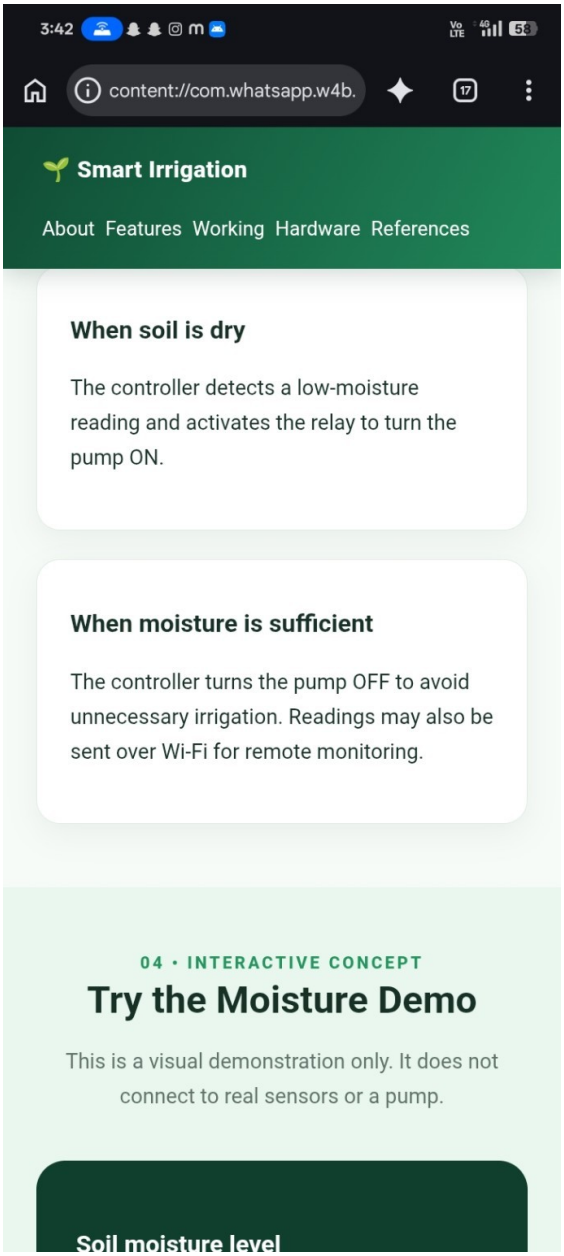


Figure 6.7: Operating cases

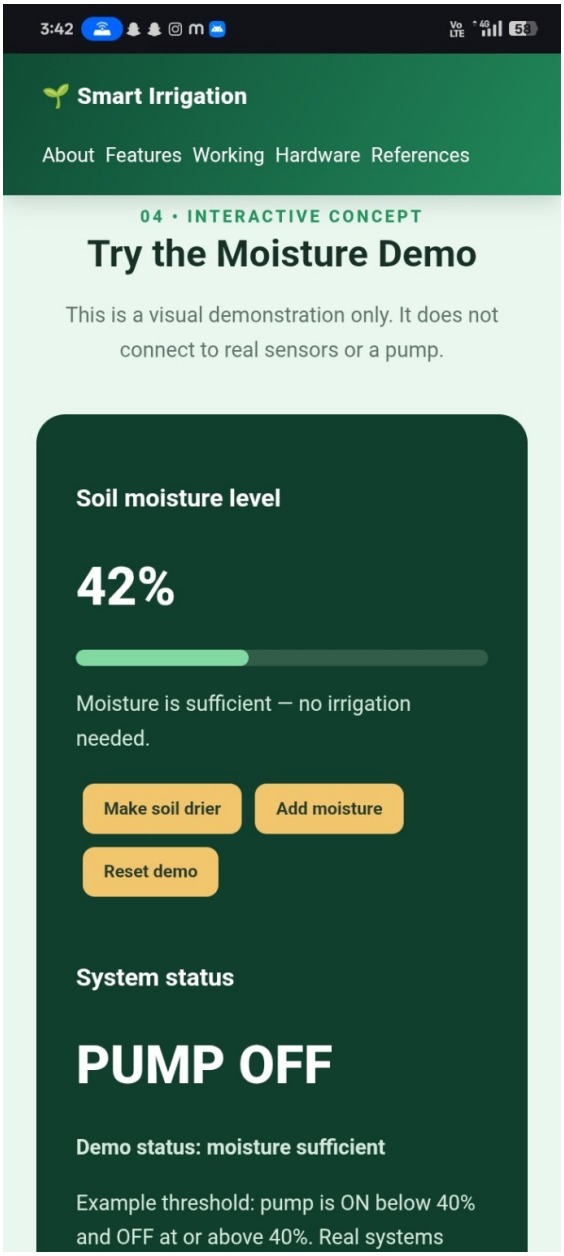


Figure 6.8: Interactive moisture demo

PROJECT OUTPUT - REQUIREMENTS AND OBJECTIVES

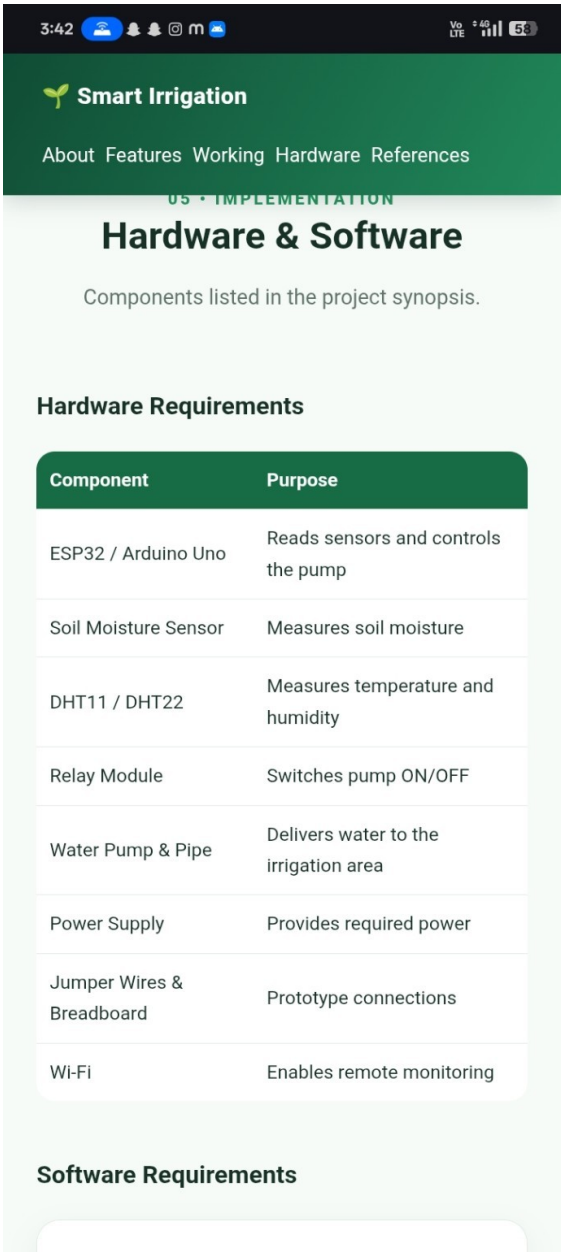


Figure 6.9: Hardware and software requirements

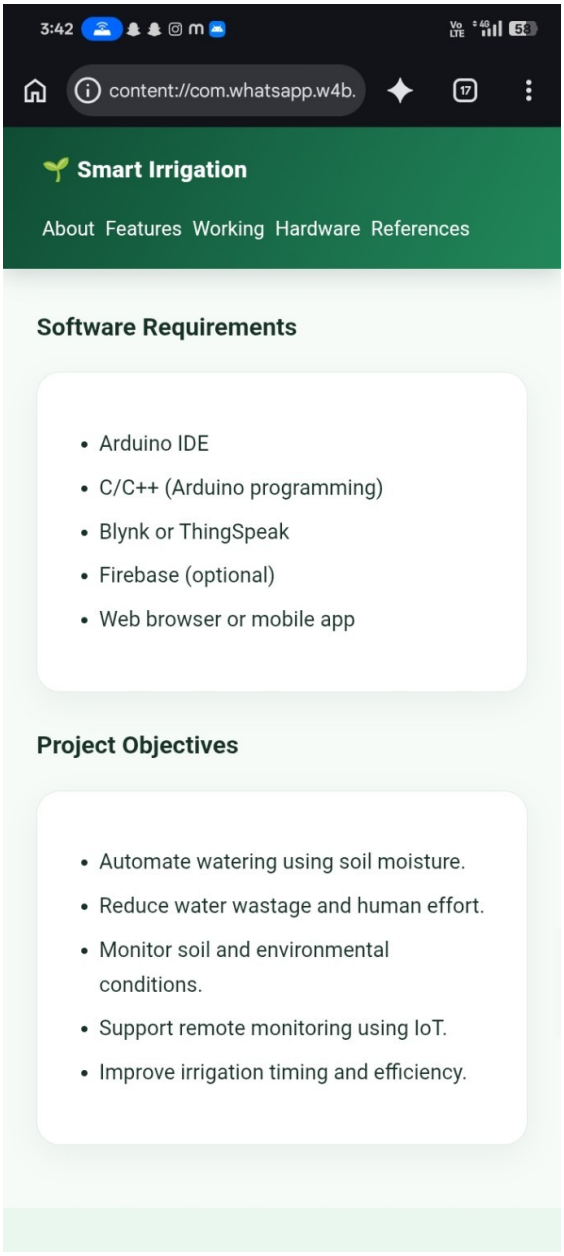


Figure 6.10: Software requirements and objectives

PROJECT OUTPUT - CONCLUSION AND REFERENCES

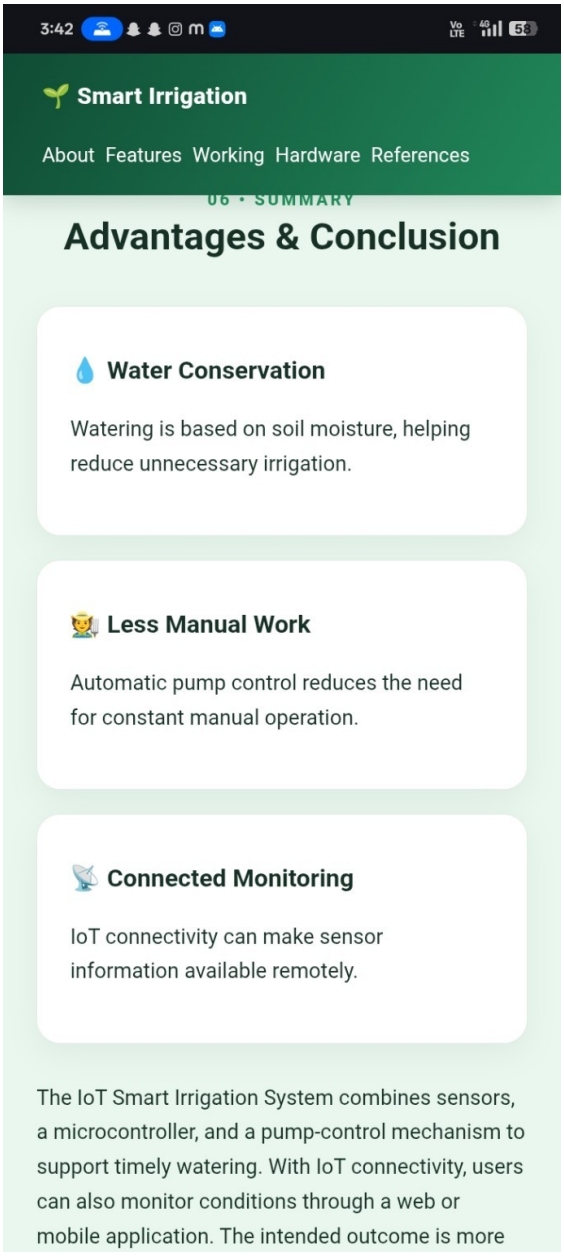


Figure 6.11: Advantages and conclusion

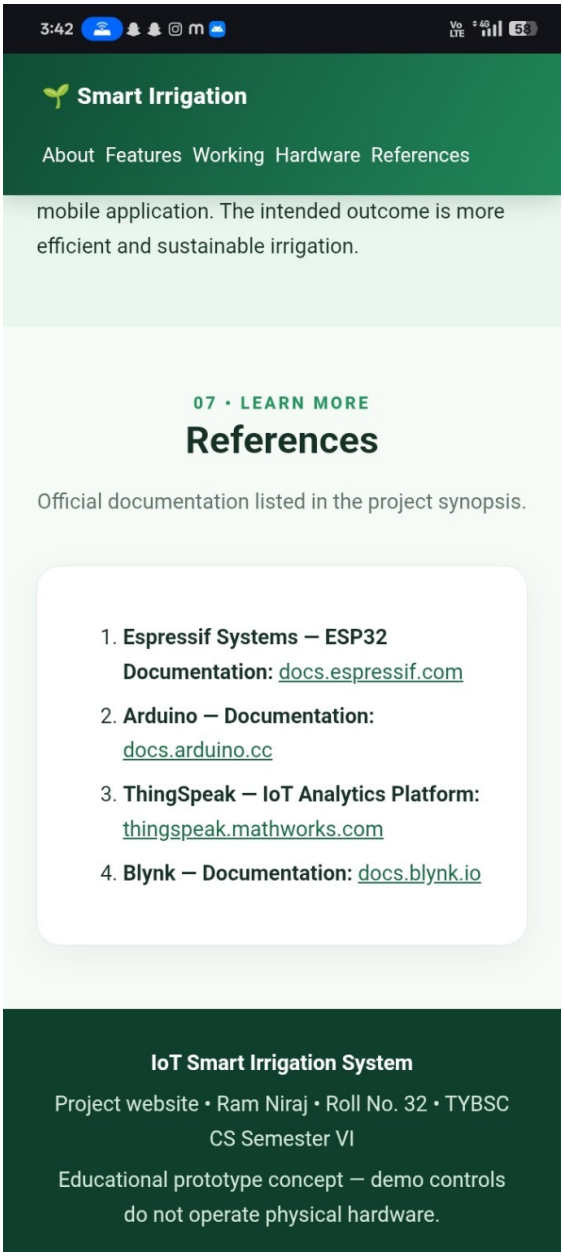


Figure 6.12: References and project footer

6.7 Test Cases and Results

Testing was planned around navigation, rendering and the deterministic behavior of the browser demo. The results below describe the supplied source logic and visible interface; they do not represent hardware test results. Based on source inspection, the demo is expected to follow the configured 40 percent threshold; no physical hardware test has been performed.

Test ID	Test action / input	Expected result	Assessment
TC-01	Open the HTML page in a modern browser	Hero section and navigation appear	Observed in supplied screenshots
TC-02	Tap About / Features / Working / Hardware / References	Page scrolls to the matching section	Anchor navigation is present in source
TC-03	Load demo at default 42%	Moisture is sufficient; pump OFF	Expected from source logic
TC-04	Select Make soil drier (25%)	Dry message; pump ON	Expected from source logic
TC-05	Select Add moisture (75%)	Moisture sufficient; pump OFF	Expected from source logic
TC-06	Select Reset demo (42%)	Value returns to 42%; pump OFF	Expected from source logic
TC-07	View on small-screen device	Content stacks and remains readable	Mobile screenshots supplied
TC-08	Check live sensor, relay and pump	No real hardware action occurs	Not implemented; explicit limitation
TC-09	Use external reference links	Browser opens selected documentation	Links are present in source
TC-10	Check data persistence after reload	Demo returns to initial default state	No persistent storage is implemented

6.8 Development Status Summary

Module	Status	Remarks
Project overview and features	Complete	Static explanatory content
Navigation and responsive layout	Complete in source	CSS and anchor links included
Interactive moisture demo	Complete in source	Client-side state only
Physical sensor connection	Not started / not included	Requires controller and sensor firmware
Relay and pump actuation	Not started / not included	Requires safe electrical implementation
Cloud / remote monitoring	Concept only	No live API or telemetry connection
Formal hardware testing	Not performed	No hardware test data supplied

CHAPTER 7 - CHALLENGES AND FUTURE SCOPE

7.1 Development Challenges

One challenge in documenting a conceptual IoT project is making sure the interface does not imply capabilities that are not connected to real hardware. This report addresses that issue by separating the current HTML/JavaScript demo from the proposed sensor-controller-pump architecture. Another challenge is presenting technical information clearly on a mobile screen, where long tables and navigation elements can become cramped.

The threshold rule is also intentionally simplified. Real soil-moisture sensors may provide raw analog or digital values that depend on sensor design, soil composition, salinity, temperature and sensor placement. Those values cannot automatically be treated as a universal percentage without calibration. Hysteresis, sampling intervals and fail-safe behavior may also be necessary to avoid rapid relay switching.

7.2 Current Limitations

- The website is a static single-page interface with client-side demo behavior only.
- The demo values are preset by buttons and are not measured from soil.
- No ESP32 or Arduino firmware, sensor connection, relay wiring or pump is included in the supplied HTML.
- No authentication, database, event history, analytics dashboard or alerting mechanism is present.
- The project has not produced measured estimates of water savings or crop performance.
- The educational threshold is illustrative and requires calibration before real use.

7.3 Future Scope

- Connect a suitable soil-moisture sensor to an ESP32 or Arduino and record calibrated readings.
- Implement embedded control logic with upper and lower thresholds or hysteresis to reduce repeated switching.
- Add a properly rated relay or driver, pump protection, electrical isolation and a manual override.
- Publish time-stamped sensor readings to a verified IoT platform and display actual values on a dashboard.
- Add temperature, humidity, weather data or crop-specific settings where useful and technically justified.
- Keep records of readings and watering events so the system can be evaluated using measurable results.
- Test usability, sensor calibration, reliability and water use on a small supervised prototype before expanding the design.

7.4 Safety and Reliability Considerations

A real pump should not be connected directly to a microcontroller output pin. Switching equipment, supply voltage and current ratings must suit the selected load, and the arrangement should use appropriate isolation and protection. Wet environments increase the importance of enclosure design, cable management and safe power handling. A physical installation should include a way to stop irrigation manually and should not rely exclusively on Wi-Fi connectivity.

CHAPTER 8 - CONCLUSION

8.1 Conclusion

The IoT Smart Irrigation System project presents an educational website that explains how sensor-based irrigation can support better watering decisions. The site introduces the problem, describes proposed components, illustrates the sensor-controller-relay-pump sequence, lists requirements and provides technical references. Its mobile-responsive design and interactive moisture demo make the project easy to explore and demonstrate.

The implemented website uses HTML for structure, CSS for presentation and responsive layout, and JavaScript for the client-side threshold demonstration. The example logic displays the pump as ON when the illustrative moisture value is below 40 percent and OFF when the value is 40 percent or higher. This is a simulation only and does not actuate physical equipment.

The main outcome is a clear front-end demonstration and a documented plan for a future physical prototype. To deliver the complete IoT functionality, a subsequent phase must implement sensor reading, calibration, controller firmware, safe pump switching, connectivity and tests with actual measurements. This distinction ensures the report accurately describes what has been completed and what remains as future work.

8.2 Learning Outcomes

- Understood the basic role of IoT sensors and actuators in a practical application.
- Organized a multi-section webpage with semantic HTML and anchor-based navigation.
- Applied CSS for colors, cards, responsive grids, flexible layouts and mobile breakpoints.
- Used JavaScript conditional logic to update a value, progress bar, message and system-status label.
- Prepared system-design diagrams, requirements tables, test cases and technical documentation.
- Learned to distinguish a visual prototype from a system connected to physical hardware.

8.3 Final Remarks

The project provides a foundation for future work in embedded systems and environmental monitoring. The website can be extended after a physical prototype is assembled and verified. Any report of water conservation, automation reliability or remote monitoring should be supported by actual measurements from that implementation.

REFERENCES

1. Espressif Systems. ESP32 Technical Documentation. *https://docs.espressif.com/*

2. Arduino. Official Arduino Documentation. *https://docs.arduino.cc/*

3. MathWorks. ThingSpeak - IoT Analytics Platform. *https://thingspeak.mathworks.com/*

4. Blynk. Blynk Platform Documentation. *https://docs.blynk.io/*

5. MDN Web Docs. HTML: HyperText Markup Language. *https://developer.mozilla.org/en-US/docs/Web/HTML*

6. MDN Web Docs. CSS: Cascading Style Sheets. *https://developer.mozilla.org/en-US/docs/Web/CSS*

7. MDN Web Docs. JavaScript Guide. *https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide*

8. Project source file. iot_smart_irrigation_website-1.html, supplied for this report. *Local project source; no public repository URL supplied.*

9. Project output captures. Mobile-browser screenshots supplied by the student on 27 October 2026. *Used as implementation evidence for website sections.*

Note: Online documentation is listed as a technical reference; the project-specific behaviors described in this report are based on the supplied HTML and screenshots.

APPENDIX A - SOURCE CODE LISTING

A.1 Introduction

The complete supplied website is a single HTML document. The following appendix provides a readable, formatted source listing of the same file for reference. The primary source remains the supplied iot_smart_irrigation_website-1.html file. The listing is formatted for print; no new hardware-control code is implied.

For readability, the HTML tags and CSS rules are placed on separate lines where practical. The JavaScript threshold function is the portion that changes the displayed demo state.

A.2 Source Listing (HTML / CSS / JavaScript)

```
001  <!DOCTYPE html>
002  <html lang="en">
003   <head>
004    <meta charset="UTF-8">
005    <meta name="viewport" content="width=device-width, initial-scale=1.0">
006    <meta name="description" content="IoT Smart Irrigation System project website by Ram Niraj, TYBSC CS Semester VI.">
007    <title>IoT Smart Irrigation System | Ram Niraj</title>
008    <style>
009     :root{--green:#176b45;--green2:#23965f;--mint:#eaf7ef;--ink:#173126;--muted:#60736a;--go
010        ld:#f2c66d;--white:#fff}
011     *{box-sizing:border-box;scroll-behavior:smooth}
012     body{margin:0;font-family:Arial,Helvetica,sans-serif;color:var(--ink);line-height:1.65;b
013        ackground:#f7fbf8}
014     header{background:linear-gradient(120deg,#104c35,#21885a);color:white;padding:18px
015        7%;position:sticky;top:0;z-index:5;box-shadow:0 4px 18px #092f1e30}
016     nav{display:flex;justify-content:space-between;align-items:center;gap:18px;flex-wrap:wra
017        p}
018     .brand{font-weight:800;font-size:1.1rem;letter-spacing:.2px}
019     nav a{color:white;text-decoration:none;margin-left:15px;font-size:.94rem}
020     nav a:hover{color:var(--gold)}
021     .hero{padding:85px 7% 78px;background:radial-gradient(circle at 85% 20%,#b7e6c8
022        0,transparent
023        28%),linear-gradient(130deg,#eaf7ef,#f9fcf9);display:grid;grid-template-columns:1.25fr
024        .75fr;gap:35px;align-items:center}
025     .eyebrow{color:var(--green2);font-weight:800;text-transform:uppercase;letter-spacing:2px
026        ;font-size:.8rem}
027     h1{font-size:clamp(2.25rem,5vw,4.4rem);line-height:1.08;margin:12px 0
028        20px;letter-spacing:-1.8px}
029     .hero p{font-size:1.1rem;color:#4b6356;max-width:650px}
030     .btn{display:inline-block;background:var(--green);color:white;padding:12px
031        20px;border-radius:10px;text-decoration:none;font-weight:700;margin:10px 8px 0 0}
032     .btn.secondary{background:white;color:var(--green);border:1px solid #b8d8c4}
033     .art{background:white;border:1px solid
034        #d9eee0;border-radius:28px;padding:28px;box-shadow:0 18px 50px
035        #164d2d15;text-align:center}
036     .plant{font-size:6rem;line-height:1.3}
037     .pill{display:inline-block;padding:6px 12px;border-radius:30px;background:var(--mint);co
038        lor:var(--green);font-size:.85rem;font-weight:bold;margin:4px}
039     section{padding:65px 7%}
040     .section-head{text-align:center;max-width:750px;margin:0 auto 35px}
041     h2{font-size:clamp(1.8rem,3vw,2.7rem);margin:0 0 12px;line-height:1.2}
042     .section-head p,.muted{color:var(--muted)}
043     .grid{display:grid;grid-template-columns:repeat(3,1fr);gap:18px}
044     .card{background:white;padding:25px;border:1px solid
045        #e1eee5;border-radius:18px;box-shadow:0 8px 24px #163e2410}
046     .icon{font-size:2rem}
047     .card h3{margin:8px 0}
048     .band{background:#eaf7ef}
049     .flow{display:flex;align-items:stretch;justify-content:center;gap:10px;flex-wrap:wrap}
050     .flow div{background:white;border:1px solid #cde6d5;padding:16px
051        18px;border-radius:12px;font-weight:700;text-align:center}
052     .arrow{align-self:center;color:var(--green2);font-size:1.5rem;font-weight:bold}
053     .two{display:grid;grid-template-columns:1fr 1fr;gap:22px}
054     ul{padding-left:22px}
055     li{margin:7px 0}
056     table{width:100%;border-collapse:collapse;background:white;border-radius:14px;overflow:h
057        idden}
```

```
    th,td{text-align:left;padding:13px 15px;border-bottom:1px solid #e3eee6}
    th{background:#176b45;color:white}
    tr:last-child td{border-bottom:0}
    .demo{background:#103f2d;color:white;border-radius:22px;padding:30px}
    .demo p{color:#d5eadc}
    .reading{font-size:2.5rem;font-weight:800;margin:8px 0}
    .track{height:12px;background:#315d49;border-radius:20px;overflow:hidden;margin:15px 0}
    .track span{display:block;height:100%;width:42%;background:#83d9a2;border-radius:20px;tr
      ansition:width .3s}
    .demo button{border:0;background:var(--gold);color:#263b2d;padding:11px
      16px;border-radius:9px;font-weight:bold;margin:5px;cursor:pointer}
    .status{font-weight:bold;color:#baf1cb}
    .refs a{color:var(--green);overflow-wrap:anywhere}
    footer{background:#103f2d;color:white;text-align:center;padding:28px 7%}
    footer p{margin:5px;color:#d5eadc}
    @media(max-width:800px){.hero{grid-template-columns:1fr;padding-top:55px}.grid{grid-temp
      late-columns:1fr 1fr}.two{grid-template-columns:1fr}nav{align-items:flex-start}nav
      div:last-child{display:flex;flex-wrap:wrap;gap:8px}nav a{margin-left:0}}
    @media(max-width:540px){.grid{grid-template-columns:1fr}section{padding:48px
      6%}.hero{padding-left:6%;padding-right:6%}th,td{padding:10px;font-size:.9rem}}
  </style>
</head>
<body>
  <header>
    <nav>
      <div class="brand">🌱 Smart Irrigation</div>
      <div>
        <a href="#about">About</a>
        <a href="#features">Features</a>
        <a href="#working">Working</a>
        <a href="#hardware">Hardware</a>
        <a href="#references">References</a>
      </div>
    </nav>
  </header>
  <main>
    <section class="hero">
      <div>
        <div class="eyebrow">IoT • Smart Agriculture • Water Conservation</div>
        <h1>Grow smarter.<br>
          <span style="color:#21885a">Save every drop.</span>
        </h1>
        <p>An IoT-based Smart Irrigation System monitors soil moisture and automatically controls watering according to the needs of the soil.</p>
        <a class="btn" href="#working">Explore how it works ↓</a>
        <a class="btn secondary" href="#about">View project</a>
        <p style="font-size:.92rem;margin-top:25px">
          <strong>Project by:</strong> Ram Niraj  |  <strong>Seat Number:</strong> 510513059<br>TYBSC CS • Semester VI</p>
      </div>
      <div class="art">
        <div class="plant">🌱💧</div>
        <h3>Intelligent Watering</h3>
        <p class="muted">Sense the soil. Make a decision. Water only when needed.</p>
        <span class="pill">Automatic control</span>
        <span class="pill">IoT monitoring</span>
        <span class="pill">Water saving</span>
      </div>
    </section>
    <section id="about">
      <div class="section-head">
        <div class="eyebrow">01 • Overview</div>
        <h2>About the Project</h2>
        <p>Modern irrigation with sensors, a controller, and connected monitoring.</p>
      </div>
      <div class="two">
        <div class="card">
        <h3>🌿 Introduction</h3>
        <p>The system uses Internet of Things (IoT) technology to monitor soil and environmental conditions. When soil becomes dry, the controller can turn the water pump ON; when enough moisture is reached, it turns the pump OFF.</p>
        </div>
        <div class="card">
        <h3>🎯 Problem Statement</h3>
        <p>Manual irrigation can lead to over-watering, under-watering, water wastage, and unnecessary electricity use. This project aims to supply water based on actual soil moisture needs.</p>
        </div>
```

```
    </div>
  </section>
  <section class="band" id="features">
    <div class="section-head">
      <div class="eyebrow">02 • What it offers</div>
      <h2>Key Features</h2>
      <p>Designed to reduce manual effort and support efficient irrigation.</p>
    </div>
    <div class="grid">
      <div class="card">
      <div class="icon">💧</div>
      <h3>Automatic Irrigation</h3>
      <p>Switches the pump ON or OFF according to the soil-moisture reading.</p>
      </div>
      <div class="card">
      <div class="icon">📊</div>
      <h3>Live Monitoring</h3>
      <p>Sensor conditions can be displayed in a mobile or web application.</p>
      </div>
      <div class="card">
      <div class="icon">🌡️</div>
      <h3>Environment Sensors</h3>
      <p>Optional temperature and humidity readings using DHT11 or DHT22.</p>
      </div>
      <div class="card">
      <div class="icon">📱</div>
      <h3>Remote Access</h3>
      <p>Wi-Fi connectivity can support remote monitoring through an IoT platform.</p>
      </div>
      <div class="card">
      <div class="icon">⚡</div>
      <h3>Less Waste</h3>
      <p>Helps avoid unnecessary watering and pump operation.</p>
      </div>
      <div class="card">
      <div class="icon">🌾</div>
      <h3>Flexible Use</h3>
      <p>Suitable in concept for gardens, farms, greenhouses, and small fields.</p>
      </div>
    </div>
  </section>
  <section id="working">
    <div class="section-head">
      <div class="eyebrow">03 • System design</div>
      <h2>How It Works</h2>
      <p>The controller checks moisture readings and makes an irrigation decision.</p>
    </div>
    <div class="flow">
      <div>🌱 Soil Moisture<br>Sensor</div>
      <span class="arrow">→</span>
      <div>🧠 ESP32 /<br>Arduino</div>
      <span class="arrow">→</span>
      <div>⚙️ Relay<br>Module</div>
      <span class="arrow">→</span>
      <div>💧 Water<br>Pump</div>
      <span class="arrow">→</span>
      <div>🌾 Irrigation</div>
    </div>
    <div class="two" style="margin-top:28px">
      <div class="card">
      <h3>When soil is dry</h3>
      <p>The controller detects a low-moisture reading and activates the relay to turn the pump ON.</p>
      </div>
      <div class="card">
      <h3>When moisture is sufficient</h3>
      <p>The controller turns the pump OFF to avoid unnecessary irrigation. Readings may also be sent over Wi-Fi for remote
monitoring.</p>
      </div>
    </div>
  </section>
  <section class="band">
    <div class="section-head">
      <div class="eyebrow">04 • Interactive concept</div>
      <h2>Try the Moisture Demo</h2>
      <p>This is a visual demonstration only. It does not connect to real sensors or a pump.</p>
    </div>
```

```
205         <div class="demo">
206           <div class="two">
207           <div>
208           <h3>Soil moisture level</h3>
209           <div class="reading">
210           <span id="moisture">42</span>%</div>
211           <div class="track">
212           <span id="bar">
213           </span>
214           </div>
215           <p id="condition">Soil is dry — irrigation needed.</p>
216           <button onclick="setMoisture(25)">Make soil drier</button>
217           <button onclick="setMoisture(75)">Add moisture</button>
218           <button onclick="setMoisture(42)">Reset demo</button>
219           </div>
220           <div>
221           <h3>System status</h3>
222           <div class="reading" id="pump">PUMP ON</div>
223           <p class="status" id="status">Demo status: irrigation needed</p>
224           <p>Example threshold: pump is ON below 40% and OFF at or above 40%. Real systems require sensor calibration and safe
electrical setup.</p>
225           </div>
226           </div>
227         </div>
228       </section>
229       <section id="hardware">
230         <div class="section-head">
231           <div class="eyebrow">05 • Implementation</div>
232           <h2>Hardware & Software</h2>
233           <p>Components listed in the project synopsis.</p>
234         </div>
235         <div class="two">
236           <div>
237           <h3>Hardware Requirements</h3>
238           <table>
239           <thead>
240           <tr>
241           <th>Component</th>
242           <th>Purpose</th>
243           </tr>
244           </thead>
245           <tbody>
246           <tr>
247           <td>ESP32 / Arduino Uno</td>
248           <td>Reads sensors and controls the pump</td>
249           </tr>
250           <tr>
251           <td>Soil Moisture Sensor</td>
252           <td>Measures soil moisture</td>
253           </tr>
254           <tr>
255           <td>DHT11 / DHT22</td>
256           <td>Measures temperature and humidity</td>
257           </tr>
258           <tr>
259           <td>Relay Module</td>
260           <td>Switches pump ON/OFF</td>
261           </tr>
262           <tr>
263           <td>Water Pump & Pipe</td>
264           <td>Delivers water to the irrigation area</td>
265           </tr>
266           <tr>
267           <td>Power Supply</td>
268           <td>Provides required power</td>
269           </tr>
270           <tr>
271           <td>Jumper Wires & Breadboard</td>
272           <td>Prototype connections</td>
273           </tr>
274           <tr>
275           <td>Wi-Fi</td>
276           <td>Enables remote monitoring</td>
277           </tr>
278           </tbody>
279           </table>
```

```
280         </div>
281         <div>
282         <h3>Software Requirements</h3>
283         <div class="card">
284         <ul>
285         <li>Arduino IDE</li>
286         <li>C/C++ (Arduino programming)</li>
287         <li>Blynk or ThingSpeak</li>
288         <li>Firebase (optional)</li>
289         <li>Web browser or mobile app</li>
290         </ul>
291         </div>
292         <h3 style="margin-top:25px">Project Objectives</h3>
293         <div class="card">
294         <ul>
295         <li>Automate watering using soil moisture.</li>
296         <li>Reduce water wastage and human effort.</li>
297         <li>Monitor soil and environmental conditions.</li>
298         <li>Support remote monitoring using IoT.</li>
299         <li>Improve irrigation timing and efficiency.</li>
300         </ul>
301         </div>
302         </div>
303       </div>
304     </section>
305     <section class="band">
306       <div class="section-head">
307         <div class="eyebrow">06 • Summary</div>
308         <h2>Advantages & Conclusion</h2>
309       </div>
310       <div class="grid">
311         <div class="card">
312         <h3>💧 Water Conservation</h3>
313         <p>Watering is based on soil moisture, helping reduce unnecessary irrigation.</p>
314         </div>
315         <div class="card">
316         <h3>🤯 Less Manual Work</h3>
317         <p>Automatic pump control reduces the need for constant manual operation.</p>
318         </div>
319         <div class="card">
320         <h3>📡 Connected Monitoring</h3>
321         <p>IoT connectivity can make sensor information available remotely.</p>
322         </div>
323       </div>
324       <p style="max-width:900px;margin:28px auto 0">The IoT Smart Irrigation System combines sensors, a microcontroller, and a
pump-control mechanism to support timely watering. With IoT connectivity, users can also monitor conditions through a web or mobile
application. The intended outcome is more efficient and sustainable irrigation.</p>
325     </section>
326     <section id="references">
327       <div class="section-head">
328         <div class="eyebrow">07 • Learn more</div>
329         <h2>References</h2>
330         <p>Official documentation listed in the project synopsis.</p>
331       </div>
332       <div class="card refs">
333         <ol>
334         <li>
335         <strong>Espressif Systems — ESP32 Documentation:</strong> <a href="https://docs.espressif.com/" target="_blank"
rel="noopener">docs.espressif.com</a>
336         </li>
337         <li>
338         <strong>Arduino — Documentation:</strong> <a href="https://docs.arduino.cc/" target="_blank"
rel="noopener">docs.arduino.cc</a>
339         </li>
340         <li>
341         <strong>ThingSpeak — IoT Analytics Platform:</strong> <a href="https://thingspeak.mathworks.com/" target="_blank"
rel="noopener">thingspeak.mathworks.com</a>
342         </li>
343         <li>
344         <strong>Blynk — Documentation:</strong> <a href="https://docs.blynk.io/" target="_blank" rel="noopener">docs.blynk.io</a>
345         </li>
346         </ol>
347       </div>
348     </section>
349   </main>
350   <footer>
```

```
    <strong>IoT Smart Irrigation System</strong>
    <p>Project website • Ram Niraj • Seat Number 510513059 • TYBSC CS Semester VI</p>
    <p>Educational prototype concept — demo controls do not operate physical hardware.</p>
  </footer>
  <script>
    function setMoisture(value){
    document.getElementById('moisture').textContent=value;
    document.getElementById('bar').style.width=value+'%';
    const dry=value<40;
    document.getElementById('condition').textContent=dry?'Soil is dry — irrigation
       needed.':'Moisture is sufficient — no irrigation needed.';
    document.getElementById('pump').textContent=dry?'PUMP ON':'PUMP OFF';
    document.getElementById('status').textContent=dry?'Demo status: irrigation needed':'Demo
       status: moisture sufficient';
    }
    setMoisture(42);
  </script>
</body>
</html>
```

APPENDIX B - PROJECT SUMMARY

Item	Details
Project title	IoT Smart Irrigation System
Student	Ram Niraj
Seat number	510513059
Programme	TYBSC Computer Science - Semester VI (as shown on the supplied project website)
Institution	R.P. Institute of Hospitality & Management, affiliated to University of Mumbai
Implemented files	One standalone HTML file with embedded CSS and JavaScript
Implemented features	Project overview, feature descriptions, working concept, hardware/software lists, objectives, references and interactive moisture simulation
Example decision threshold	Pump ON below 40%; pump OFF at or above 40%
Not implemented	Physical sensor, relay and pump connections; controller firmware; live IoT telemetry; database
Future direction	Build and test a safe sensor-controller-pump prototype and connect real readings to a monitoring interface

End of Report